

Nexora Technologies Pvt. Ltd.

Software Development Coding Standards

Document ID	ENG-CS-001
Department	Development
Version	1.0
Effective Date	30 June 2026
Approved By	Daniel KC (Chief Technology Officer)

1. Purpose

This document establishes coding standards for all software development activities at Nexora Technologies Pvt. Ltd. These standards ensure consistency, maintainability, security, readability, and collaboration.

2. Scope

These standards apply to:

- Backend Development
- Frontend Development
- Full Stack Development
- APIs
- Mobile Applications
- Internal Tools
- Automation Scripts

3. General Principles

Developers should write code that is:

- Readable
- Maintainable
- Secure
- Efficient
- Reusable
- Well documented

Code clarity should always be preferred over cleverness.

4. Naming Conventions

Variables:

employeeCount
customerAddress

Functions:

calculateSalary()
sendInvoiceEmail()
validateUser()

Classes:

EmployeeService
AuthenticationManager

Constants:

MAX_LOGIN_ATTEMPTS
DEFAULT_TIMEOUT
API_VERSION

5. File Organization

Each file should have one clear responsibility.

Recommended structure:

controllers/
services/
models/
repositories/
middlewares/
utilities/
config/
tests/

Files should generally remain under 500 lines whenever possible.

6. Formatting Standards

- 4-space indentation
- UTF-8 encoding
- Unix (LF) line endings
- Maximum line length: 120 characters
- One statement per line
- Remove trailing whitespace

7. Comments

Comments should explain WHY, not WHAT.

Good:

Cache results to reduce database load.

Avoid:

Increment i

8. Error Handling

Always handle exceptions gracefully.

- Log unexpected errors.
- Never expose stack traces to users.
- Display friendly error messages.

9. Logging

Log:

- Authentication events
- System errors
- API failures
- Database failures

Never log:

- Passwords
- API Keys
- Tokens
- Sensitive customer data

10. Security

- Validate all input.
- Sanitize output.
- Use parameterized SQL queries.
- Encrypt sensitive information.
- Store secrets in environment variables.
- Follow least-privilege access.

11. Database Standards

- Use foreign keys.
- Normalize data where appropriate.
- Table names should be plural.

Examples:

Employees
Projects
Departments
Invoices

12. API Standards

Use REST principles.

Methods:

GET
POST
PUT

PATCH
DELETE

Response Example:

```
{  
  "success": true,  
  "message": "Operation completed.",  
  "data": {}  
}
```

13. Version Control

Git is the official version control system.

Developers must:

- Use feature branches
- Submit Pull Requests
- Receive code review approval

Branch examples:

feature/login-page
bugfix/payment-error
release/v2.3

14. Commit Messages

Good:

Add employee authentication module

Fix invoice calculation bug

Improve dashboard performance

Avoid vague messages like:

Update
Changes
Fix

15. Code Reviews

Every Pull Request must:

- Build successfully
- Pass automated tests
- Follow coding standards
- Receive reviewer approval

16. Testing

Developers should write:

- Unit Tests
- Integration Tests
- API Tests

Minimum code coverage target: 80%

17. Documentation

Every major feature should include:

- Purpose
- Configuration
- Dependencies
- Architecture overview
- Deployment notes

18. Performance

Developers should:

- Minimize database queries
- Cache frequently accessed data
- Remove duplicate code
- Optimize API responses

19. Third-Party Libraries

Before adding dependencies:

- Verify licensing
- Check maintenance status
- Review security vulnerabilities

Remove unused libraries promptly.

20. Compliance

Failure to follow these standards may result in:

- Code review rejection
- Delayed deployment
- Additional review

Repeated violations may be escalated to the Development Team Lead and CTO.

Document Control

Document Owner	Development Department
Approved By	Daniel KC (Chief Technology Officer)
Review Cycle	Every 12 Months
Classification	Internal Use Only

Version History

Version	Date	Description	Approved By
1.0	30-Jun-2026	Initial Release	Daniel KC